

ShadowBoard -AI

AI Powered Multi-Agent Decision Intelligence Platform

Team Name: Quick Bytes

Team Members: Mithin Kumar G

Hruday P N

Akanksha K S

College : Bangalore Institute of Technology

Date: 2/7/2026

1. Executive Summary

ShadowBoard AI is an AI-powered decision intelligence platform that gives founders, product leaders, and executive teams instant access to a virtual boardroom of domain-specialist AI agents. Rather than relying on a single generic chatbot response, the platform convenes seven distinct AI personas — a CFO, a Product Strategy lead, an Engineering lead, a Marketing lead, a Legal and Compliance advisor, a Risk analyst, and a Moderator who synthesizes their views — to analyze a real business question against the organization's own uploaded documents and return one coherent, evidence-backed recommendation.

Organizations that need to make consequential decisions — pricing changes, market expansion, fundraising timing, product trade-offs — typically scatter that analysis across spreadsheets, disconnected conversations, and individual judgment calls. Critical context buried in contracts, financial statements, and research reports rarely reaches every stakeholder in time, and recommendations often reflect whoever spoke last rather than the strongest evidence. ShadowBoard AI addresses this by grounding every agent's analysis in retrieval over the company's own documents, then forcing explicit disagreement and resolution between agents before a final answer is produced.

The platform's core innovation is its two-round Agent-to-Agent (A2A) debate protocol: each domain agent first analyzes the question independently, then reviews its peers' conclusions and is given the opportunity to revise its own position before the Moderator Agent synthesizes a single recommendation, an explicit confidence score, and a list of unresolved risks. This mirrors how a real executive board reaches consensus — through independent expertise, visible disagreement, and deliberate synthesis — rather than simply averaging outputs.

Technically, ShadowBoard AI combines a FastAPI backend, a React and TypeScript frontend, PostgreSQL for relational data, Qdrant as a vector store for semantic retrieval, Google Gemini for both embeddings and generation, and Celery with Redis for asynchronous orchestration of long-running multi-agent workflows. The result is a production-shaped system — authenticated, observable, and deployed on public infrastructure — that demonstrates how multi-agent AI architectures can be applied to real organizational decision-making rather than single-turn question answering.

By combining retrieval-augmented generation, structured multi-agent debate, and a polished operator-facing dashboard, ShadowBoard AI delivers measurable business impact: faster time-to-decision, recommendations that are traceable back to source documents, and a transparent record of which expert perspectives agreed or disagreed and why.

2. Introduction

2.1 What ShadowBoard AI Is

ShadowBoard AI is a web-based platform where a user uploads the documents relevant to a business decision — financial statements, product roadmaps, contracts, market research — and then asks a plain-language question such as "Should we raise prices by 15 percent next quarter?"

or "Should we expand into the EU market next quarter?" The platform processes those documents into a searchable knowledge base, dispatches the question to seven specialized AI agents that each retrieve the context relevant to their domain, runs a structured debate between them, and returns a final recommendation together with the full reasoning trail.

2.2 Why Organizations Struggle With Strategic Decision Making

Strategic decisions rarely fail because the relevant information doesn't exist — they fail because that information is fragmented across finance, product, engineering, legal, and marketing functions, and no single person can hold all of it in view at once. Manually reviewing every contract, spreadsheet, and report before a decision is made is slow, and the resulting recommendation is only as good as whichever stakeholder happened to read the most relevant document.

2.3 How AI Agents Improve Collaborative Decision Making

Multi-agent AI systems make it possible to assign each domain a dedicated, consistently-prompted expert perspective that always grounds its output in retrieved evidence rather than general knowledge alone. By having agents review each other's conclusions in a debate round before a synthesis step, ShadowBoard AI produces a recommendation that has already absorbed cross-functional pushback — the kind of scrutiny a real board applies — instead of a single unreviewed answer.

3. Problem Statement

Teams making high-stakes business decisions today face a consistent set of structural problems:

- Fragmented decision making — financial, product, legal, and market perspectives are gathered in separate conversations and rarely reconciled before a decision is finalized.
- Manual document analysis — contracts, financial statements, and research reports must be read individually by each stakeholder, which does not scale as document volume grows.
- Information overload — decision-makers are handed large volumes of unstructured material with no efficient way to surface the passages that actually bear on the question at hand.
- Lack of collaboration — cross-functional input is often gathered sequentially rather than debated, so conflicts between, for example, a Legal concern and a Marketing opportunity go unexamined.
- Delayed decisions — assembling the right people and documents for a single decision can take days, by which point the underlying market conditions may have changed.
- Inconsistent recommendations — the same question asked of different stakeholders on different days can produce contradictory guidance, with no record of why opinions diverged.

ShadowBoard AI is built directly against these six failure modes: it keeps every agent grounded in the same underlying document set, runs analysis in parallel rather than sequentially, and produces one synthesized, timestamped recommendation with a visible reasoning trail.

4. Objectives

1. Provide a single platform where business documents can be uploaded once and reused across every future decision query.
2. Ground every AI-generated recommendation in retrieval over the organization's own documents rather than unverified model knowledge.
3. Model a realistic executive boardroom by dividing analysis across seven distinct functional perspectives.
4. Surface genuine disagreement between domain agents instead of silently averaging their conclusions.
5. Produce a final recommendation accompanied by an explicit, numeric confidence score.
6. Maintain a complete, auditable transcript of each agent's reasoning across both rounds of debate.
7. Deliver decision turnaround in minutes rather than the days a manual cross-functional review typically requires.
8. Secure every document and decision behind authenticated, per-user access control.
9. Design the system to scale from a single hackathon demo to a production-grade deployment without architectural rework.
10. Keep the entire reasoning pipeline observable, with structured logging and metrics at every stage.

5. Proposed Solution

ShadowBoard AI is delivered as a full-stack web platform in which every stage of the decision-making journey — from raw document to final recommendation — is handled end-to-end by the system itself.

5.1 Document Upload and Processing

Users upload business documents — PDF reports, spreadsheets, and plain text files — through the dashboard. Each document is parsed and its full text extracted using format-specific handlers, then normalized into a single text stream ready for indexing.

5.2 Knowledge Indexing

Extracted text is split into overlapping, sentence-boundary-aware chunks, converted into vector embeddings using Google Gemini's embedding model, and stored in a Qdrant vector collection alongside metadata identifying the source document and owning project. This gives every subsequent query fast, semantically relevant retrieval scoped to the correct project.

5.3 Multi-Agent Analysis

When a user asks a decision question, the Moderator Agent first decomposes it into domain-specific sub-questions for each of the six specialist agents. Every specialist independently retrieves

the document passages most relevant to its sub-question and produces a structured analysis: a summary, a recommendation, a confidence score, identified risks, and citations back to source material.

5.4 Debate

Each agent then reviews the other agents' first-round conclusions. If a peer has raised a consideration material to its own domain, the agent revises its summary, recommendation, confidence, or risk list accordingly; otherwise it restates its original position. This second round is the platform's Agent-to-Agent debate — disagreement is made explicit and addressed before any answer reaches the user.

5.5 Moderation and Final Recommendation

The Moderator Agent synthesizes both rounds into one final recommendation, a paragraph explaining how conflicting expert input was weighed, an overall confidence score, a consolidated list of key risks, and an explicit record of any dissenting views that were not fully resolved.

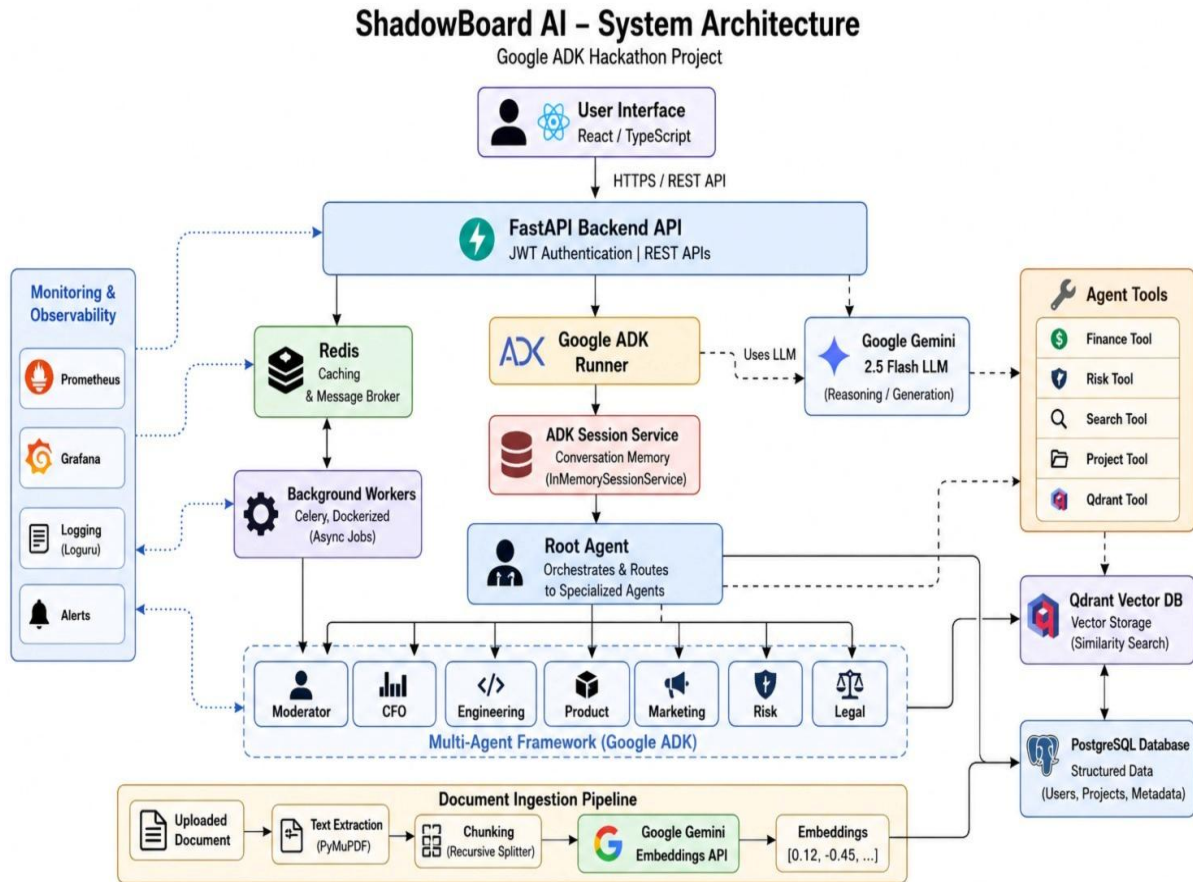
5.6 Dashboard Delivery

The final recommendation, confidence score, risk summary, and full agent transcript are streamed to the user's dashboard in near real time, giving decision-makers both the answer and full visibility into how it was reached.

6. System Architecture

ShadowBoard AI follows a layered architecture that separates presentation, orchestration, retrieval, and reasoning into distinct, independently scalable components.

- React Frontend — authentication, document upload, decision requests, live dashboard
- FastAPI Backend — the single REST entry point for all client traffic
- Authentication Layer — JWT-based identity verification on every protected route
- Document Processing — format-aware text extraction from PDF, spreadsheet, and text sources
- Chunking — sliding-window, sentence-boundary-aware segmentation of extracted text
- Gemini Embeddings — vectorization of each chunk for semantic retrieval
- Qdrant Vector Database — persistent, project-scoped semantic index
- AI Agent Framework — seven specialized agents built on a shared base-agent abstraction
- Debate Engine — the two-round Agent-to-Agent protocol coordinating peer review
- Recommendation Engine — Moderator-driven synthesis into one final answer
- Dashboard — real-time delivery of the recommendation, confidence, risks, and transcript



6.1 Frontend Layer

Built with React, TypeScript, and Vite, the frontend handles authentication state, document upload, decision submission, and a live decision view that polls for status until the pipeline completes. Tailwind CSS drives styling, Zustand manages authentication state, and TanStack Query manages server state and polling.

6.2 FastAPI Backend

The backend exposes a versioned REST API covering authentication, document management, and decision orchestration. It validates every request, persists state to PostgreSQL, and dispatches long-running work to Celery so that API responses remain fast even while multi-agent analysis is in progress.

6.3 Authentication

Every protected endpoint requires a bearer JWT issued at registration or login. Passwords are never stored in plain text; they are hashed with bcrypt before persistence.

6.4 Document Processing and Chunking

Uploaded files are routed to a format-specific extractor, then split into overlapping chunks sized to balance retrieval precision against context completeness.

6.5 Embeddings and Vector Storage

Each chunk is embedded with Gemini's embedding model and upserted into a dedicated Qdrant collection with project-level metadata, enabling retrieval to be scoped precisely to the documents relevant to a given decision.

6.6 AI Agent Framework, Debate Engine, and Recommendation Engine

A shared base-agent class provides retrieval, LLM invocation with automatic retry, and structured JSON output handling to every specialist agent. The debate engine coordinates the two analysis rounds, and the recommendation engine — the Moderator Agent's synthesis step — produces the final, board-level answer.

6.7 Dashboard

The dashboard renders the completed decision: the final recommendation, a numeric confidence score, a consolidated risk list, and the full per-agent debate transcript, giving users both the conclusion and the reasoning behind it.

7. Technology Stack

ShadowBoard AI is built on a modern, production-oriented technology stack chosen for reliability, asynchronous performance, and straightforward cloud deployment.

Layer	Technology
Frontend	React, TypeScript, Vite
Backend	FastAPI (Python 3.11), Uvicorn ASGI server
Database	PostgreSQL, accessed via SQLAlchemy (async) with Alembic migrations
Vector Database	Qdrant (self-hosted via Docker or Qdrant Cloud)
Authentication	JWT (python-jose) with bcrypt password hashing (passlib)
Caching / Message Broker	Redis (self-hosted via Docker or Upstash for cloud deployment)
Background Tasks	Celery workers for document ingestion and decision-pipeline orchestration
AI Models	Google Gemini for generation (gemini-2.5-pro) and embeddings (text-embedding-004), accessed via the google-genai SDK
Deployment	Docker and Docker Compose for local and production orchestration; Render for backend and worker services; Vercel for the frontend; Neon for managed PostgreSQL
Monitoring	Prometheus metrics collection with Grafana dashboards; Loguru for structured application logging
Testing	Pytest and pytest-asyncio, with httpx for asynchronous API test clients

Layer	Technology
Programming Languages	Python (backend, agents, orchestration); TypeScript (frontend)
Key Libraries	SQLAlchemy, Alembic, Pydantic, Tenacity, pdfplumber, pandas, Zustand, TanStack Query, Axios, Recharts, Framer Motion, Lucide
Cloud Services	Neon (PostgreSQL), Upstash (Redis), Qdrant Cloud (vector storage), Render (compute), Vercel (static hosting / CDN)

8. Core Modules

The platform is composed of fifteen cooperating modules, each with a single, well-defined responsibility.

User Authentication

Handles registration, login, password hashing, and JWT issuance and verification, forming the security boundary for every other module.

Project Management

Groups documents and decisions under a project, scoping retrieval and access to the correct organizational context.

Document Upload

Accepts PDF, spreadsheet, and text files through a multipart API, storing the original file and creating a tracked document record.

Document Extraction

Converts each supported file format into plain text using format-specific parsers, capturing page or sheet counts for traceability.

Chunking

Splits extracted text into overlapping, sentence-aware segments sized for reliable embedding and retrieval.

Embedding Generation

Converts each chunk into a dense vector representation using Gemini's embedding model, in batches for efficiency.

Vector Storage

Persists chunk vectors and metadata in Qdrant, partitioned by project so retrieval never crosses project boundaries.

Semantic Search

Executes similarity search against Qdrant for a given query, returning the most relevant passages with relevance scores.

Multi-Agent Framework

A shared base-agent abstraction providing retrieval, LLM invocation, retry handling, and structured JSON parsing to all seven agents.

Debate Protocol

Coordinates the two-round Agent-to-Agent exchange — independent analysis, then peer-aware revision — that underlies every decision.

Decision Engine

Orchestrates the full pipeline for a submitted question: sub-question planning, parallel analysis, debate, and synthesis.

Recommendation Generator

The Moderator Agent's synthesis step, producing the final recommendation, confidence score, and risk summary.

Dashboard

The user-facing interface for uploading documents, submitting questions, and reviewing completed decisions.

Monitoring

Exposes application metrics for Prometheus scraping and structured logs for operational visibility.

Deployment

Containerized services orchestrated through Docker Compose locally and distributed across managed cloud platforms in production.

9. AI Workflow

Every decision follows the same deterministic pipeline, ensuring consistent, auditable behavior from raw document to final recommendation.

- ▶ Document Upload — the user submits a business document through the dashboard
- ▶ Extraction — the document's full text is extracted using a format-specific parser
- ▶ Chunking — the text is divided into overlapping, retrieval-sized segments
- ▶ Embeddings — each chunk is vectorized using Gemini's embedding model
- ▶ Qdrant — the resulting vectors are indexed in the project's vector collection
- ▶ Retrieval — on a decision request, each agent retrieves the passages relevant to its sub-question
- ▶ Agent Analysis — all six specialist agents independently analyze their sub-question in parallel
- ▶ Debate — agents review peer conclusions and revise their position where warranted
- ▶ Consensus — the Moderator weighs all evidence and resolves remaining disagreement
- ▶ Recommendation — a final answer, confidence score, and risk summary are persisted and returned

This pipeline runs asynchronously through Celery so that a user can submit a question and continue working while the platform completes what is, in effect, a full round of expert deliberation in the background.

10. Multi-Agent System

ShadowBoard AI's seven agents each operate as an independent specialist, sharing a common analytical structure — retrieval, structured reasoning, and JSON output — while differing entirely in domain focus and decision criteria.

Moderator Agent

Role	Orchestrates the boardroom rather than analyzing any single domain. It decomposes the user's question into a tailored sub-question for each specialist, and after both analysis rounds, synthesizes every agent's output into one final recommendation.
Decision Criteria	Coherence across all six domains; the quality of the final synthesis and confidence assignment.
Inputs	All six specialist agents' first- and second-round outputs, plus the original user question.
Outputs	The final recommendation, supporting reasoning, an overall confidence score, key risks, and any dissenting views.
Importance	The Moderator is what turns six independent opinions into one board-level decision — without it the platform would only produce fragmented, uncoordinated analysis.

CFO Agent

Role	Evaluates the financial dimension of the decision: ROI, burn rate, budget impact, profitability, cash runway, and investment risk.
Decision Criteria	Weights figures explicitly drawn from company financial documents over unsupported projections, and states plainly when financial data is missing.
Inputs	Retrieved financial context relevant to the question, plus the Moderator's tailored sub-question.
Outputs	A financial summary, recommendation, confidence score, financial risks, and citations to source figures.
Importance	Financial viability is a precondition for almost every strategic decision, making this agent's assessment foundational to the board's final answer.

Product Strategy Agent

Role	Evaluates roadmap fit, customer needs, feature prioritization, and product-market fit.
Decision Criteria	Weights documented customer feedback more heavily than internal assumptions.
Inputs	Retrieved product and customer-feedback context relevant to the question.
Outputs	A product-focused summary, recommendation, confidence score, risks, and citations.
Importance	Ensures decisions remain grounded in what customers and the roadmap actually support, not just financial or technical feasibility.

Engineering Agent

Role	Assesses technical feasibility, system architecture impact, scalability, security implications, and realistic delivery timelines.
Decision Criteria	Flags unrealistic timelines and technical risk directly rather than deferring to optimistic assumptions.
Inputs	Retrieved technical and architectural context relevant to the question.
Outputs	A technical summary, recommendation, confidence score, risks, and citations.
Importance	Prevents the board from recommending a direction that is financially or strategically attractive but not technically achievable in the stated timeframe.

Marketing Agent

Role	Analyzes brand positioning, customer acquisition cost, pricing strategy, competitor positioning, and market timing.
Decision Criteria	Grounds recommendations in the company's own market research and competitor analysis documents.
Inputs	Retrieved market and competitive context relevant to the question.
Outputs	A market-focused summary, recommendation, confidence score, risks, and citations.

Importance	Brings external market reality into a decision process that could otherwise be dominated by internal financial or technical considerations.
-------------------	---

Legal Agent

Role	Reviews contracts, regulatory exposure, privacy obligations, licensing terms, and general compliance risk.
Decision Criteria	Treats ambiguous contract language as a risk to flag rather than an assumption of compliance, and is explicit that it does not substitute for a licensed attorney.
Inputs	Retrieved contract and compliance context relevant to the question.
Outputs	A compliance-focused summary, recommendation, confidence score, legal risks, and citations.
Importance	Surfaces regulatory and contractual exposure early, before a decision is acted on rather than after.

Risk Agent

Role	Synthesizes financial, operational, technical, legal, and strategic risk signals across the full document set rather than a single domain.
Decision Criteria	Assigns each identified risk an explicit likelihood and impact rating.
Inputs	Retrieved cross-domain context spanning all available documents.
Outputs	A consolidated risk summary, recommendation, confidence score, structured risk list, and citations.
Importance	Provides the board with a single, cross-cutting view of what could go wrong, complementing the domain-specific risk notes raised by other agents.

11. Backend Design

11.1 FastAPI and REST APIs

The backend is built on FastAPI, exposing versioned REST endpoints for authentication, document management, and decision orchestration. FastAPI's async support allows the API to remain responsive while document ingestion and multi-agent analysis run in the background.

11.2 Authentication and JWT

Registration and login endpoints issue signed JWTs containing the authenticated user's identity. Every protected route verifies the bearer token on each request and resolves it back to a database-backed user record before allowing access.

11.3 Celery and Redis

Document ingestion and decision-pipeline execution are dispatched as Celery tasks, with Redis serving as both the message broker and result backend. This keeps API responses fast even when a task involves multiple sequential Gemini calls across seven agents.

11.4 Database

PostgreSQL stores users, projects, documents, and decisions through an async SQLAlchemy ORM, with Alembic managing schema migrations so the production schema stays synchronized with the codebase on every deployment.

11.5 Qdrant Integration

A dedicated Qdrant client module manages collection creation, vector upserts, and project-scoped similarity search, isolating all vector-database logic behind a small, well-defined interface used by both the ingestion pipeline and the agents.

11.6 Configuration

All environment-specific settings — database URLs, API keys, model names, CORS origins — are centralized in a single typed settings object, loaded from environment variables and validated at startup.

11.7 Logging

Structured logging is handled through Loguru, writing to both standard output and rotating log files, giving operators a consistent view of application behavior across environments.

11.8 Monitoring

Prometheus instrumentation exposes request-level metrics at a dedicated endpoint, which Grafana dashboards consume to visualize latency, throughput, and error rates in real time.

11.9 Error Handling

LLM calls are wrapped in automatic retry logic with exponential backoff to absorb transient provider failures, and every agent falls back to a well-formed error response rather than propagating a raw exception if output parsing fails.

12. Frontend Design

12.1 React and TypeScript

The frontend is a strictly typed React application built with Vite, giving fast local iteration and a small, efficient production bundle.

12.2 Dashboard

The central dashboard lets a user upload documents to a project and monitor their processing status, and submit new decision questions to the board.

12.3 Authentication

A dedicated login page authenticates against the backend and stores the resulting JWT, gating access to the dashboard and decision views.

12.4 Decision Page

Each decision has a dedicated page that polls the backend at short intervals until the pipeline reaches a terminal status, then renders the final recommendation, confidence score, risk list, and full agent debate transcript.

12.5 Upload Interface

A lightweight upload control accepts supported file types and reflects each document's live processing status — pending, processed, or failed — alongside its chunk count.

12.6 Visualization

Agent confidence scores and risk breakdowns are rendered visually to help users assess the strength of a recommendation at a glance.

12.7 State Management

Zustand manages lightweight global authentication state, while TanStack Query manages all server state, caching, and polling, keeping data-fetching logic out of individual components.

13. Database Design

The relational schema is intentionally compact, with four core entities capturing everything the platform needs to track.

Entity	Purpose
Users	Stores authenticated accounts, including hashed passwords and profile information.
Projects	Groups documents and decisions under a named workspace owned by a user.
Documents	Tracks each uploaded file's storage location, type, processing status, and chunk count.
Decisions	Stores each submitted question, its status, the final recommendation, confidence score, full agent transcript, and risk summary.

13.1 Relationships

- A User owns zero or more Projects.
- A Project contains zero or more Documents and zero or more Decisions.
- A Decision's agent transcript and risk summary are stored as structured JSON, preserving the full multi-round debate without requiring additional tables.

13.2 Data Flow

When a document is uploaded it is written to the Documents table with a pending status, then updated to processed or failed once ingestion completes. When a decision is submitted it is written with a pending status, updated to running once the Celery task begins, and finally updated to completed or failed with the full recommendation payload attached.

14. Deployment

ShadowBoard AI is deployed as a fully containerized system, with a local Docker Compose configuration mirroring the production topology so behavior stays consistent across environments.

14.1 Containerization

The backend and Celery worker are each built from a lightweight Python container image, while the frontend is compiled to static assets and served from a minimal Nginx image.

14.2 Managed Data Services

Production data services are fully managed: PostgreSQL runs on Neon's serverless platform, Redis runs on Upstash, and the vector store runs on Qdrant Cloud — eliminating self-managed database operations entirely.

14.3 Application Hosting

The FastAPI backend and Celery worker are deployed as independent services on Render, each connected to the same source repository and Redis instance, while the React frontend is deployed to Vercel's global edge network.

14.4 Production Environment

Environment-specific configuration — database connection strings, API keys, and CORS origins — is managed entirely through platform environment variables, keeping secrets out of the codebase and allowing configuration to differ cleanly between local and production environments.

14.5 Scalability

Because the backend, worker, and frontend are deployed as independent services, each can be scaled horizontally on its own — for example, adding additional Celery worker instances during periods of heavy decision volume — without redeploying the rest of the system.

15. Security Features

- JWT Authentication — every protected endpoint requires a valid, signed bearer token tied to an authenticated user.
- Password Hashing — user passwords are hashed with bcrypt before storage; plain-text passwords are never persisted.
- API Security — request payloads are validated against strict schemas before any business logic executes.
- Environment Variables — all credentials and secrets (database URLs, API keys, JWT signing secret) are supplied through environment configuration rather than hardcoded values.
- Protected Routes — document and decision endpoints resolve and enforce the requesting user's identity before returning any data.
- CORS — cross-origin access is restricted to an explicit allow-list of trusted frontend origins.
- Role-Based Considerations — the data model scopes every project, document, and decision to its owning user, providing the foundation for finer-grained role-based access as the platform grows.

16. Testing

16.1 Unit Testing

Core application behavior, including health checks, is covered by pytest-based unit tests that run against the FastAPI application directly.

16.2 API Testing

Authentication flows — registration and login — are tested end-to-end using an asynchronous HTTP client against the live application instance, verifying both success and expected failure paths.

16.3 Integration Testing

The full pipeline — upload, ingestion, embedding, multi-agent analysis, and synthesis — is verified through manual and scripted end-to-end runs against a live environment before each deployment.

16.4 Deployment Verification

Each production deployment is verified through a health-check endpoint and a full manual pass through registration, upload, and decision submission before being considered live.

16.5 Monitoring and Logging

Prometheus metrics and structured Loguru logs give ongoing visibility into request latency, task throughput, and failure rates after deployment, supplementing pre-release testing with continuous operational insight.

17. Key Features

- Multi-format document ingestion covering PDF, spreadsheet, and plain-text sources
- Semantic search over uploaded documents powered by Gemini embeddings and Qdrant
- Seven distinct AI agents, each with its own domain expertise and decision criteria
- A genuine two-round Agent-to-Agent debate protocol rather than single-pass analysis
- Automatic sub-question decomposition tailored to each agent's domain
- Moderator-driven synthesis into one final, board-level recommendation
- Explicit numeric confidence scoring on every recommendation
- A consolidated, likelihood-and-impact-rated risk summary
- Full transparency into dissenting agent views that were not fully resolved
- A complete, persisted transcript of every agent's reasoning across both debate rounds
- JWT-secured, per-user project isolation
- Asynchronous background processing so long-running analysis never blocks the API
- A live, auto-refreshing dashboard that reflects decision status in real time
- Production-grade observability through Prometheus metrics and structured logging
- A fully containerized deployment that runs identically in local and cloud environments

18. Advantages

- Recommendations are grounded in the organization's own documents rather than generic model knowledge
- Cross-functional disagreement is surfaced and addressed before a decision reaches the user
- Decision turnaround measured in minutes rather than days of manual review
- A consistent, repeatable decision process regardless of which stakeholders are available
- Full auditability — every recommendation can be traced back to specific agent reasoning and source citations
- Modular agent design makes it straightforward to add new domain experts
- Asynchronous architecture keeps the platform responsive even during heavy multi-agent workloads
- Cloud-native deployment removes the operational burden of managing databases directly
- Confidence scoring gives decision-makers a calibrated sense of how strong a recommendation is
- Risk summaries make trade-offs explicit rather than implicit
- Project-based isolation keeps each organization's documents and decisions cleanly separated
- The platform scales from a local demo to a public production deployment without architectural changes

19. Challenges Faced

19.1 Technical Challenges

Coordinating seven independent agents into a single coherent output required careful design of the shared base-agent abstraction, structured JSON contracts between agents, and defensive parsing to handle occasional malformed model output gracefully rather than failing the entire pipeline.

19.2 Deployment Challenges

Splitting the system across managed services — Neon, Upstash, Qdrant Cloud, Render, and Vercel — required careful alignment of connection strings, TLS requirements, and cross-origin configuration so that every component could reach the others reliably in production.

19.3 AI Integration Challenges

Keeping every agent's output in a strictly parseable JSON format, despite the inherent variability of LLM responses, required explicit prompt constraints and fallback handling for any response that failed to parse cleanly.

19.4 Scaling Challenges

Running six agents through two full debate rounds for a single decision is inherently latency-heavy; the pipeline was designed around asynchronous, parallel execution and background task queuing specifically to keep this workload from blocking the rest of the platform.

20. Future Enhancements

- Additional specialist agents covering domains such as operations, HR, and sustainability
- Voice-based interaction for submitting and reviewing decisions hands-free
- Enterprise integrations with Slack for decision notifications and approvals
- Native integration with Notion for pulling and syncing supporting documentation
- Google Drive integration for direct document ingestion without manual upload
- Microsoft Teams integration for boardroom-style collaborative review
- Advanced analytics dashboards tracking decision outcomes over time
- Predictive intelligence that flags decisions likely to require future revisiting
- Real-time collaborative review, allowing multiple stakeholders to discuss a decision as it is generated
- A dedicated mobile application for reviewing and acting on recommendations on the go

21. Conclusion

ShadowBoard AI demonstrates that multi-agent AI systems can move beyond single-turn question answering into genuine collaborative reasoning. By grounding seven specialized agents in an organization's own documents and forcing explicit debate before synthesis, the platform produces recommendations that reflect the kind of scrutiny a real executive board would apply — not just what a single model happens to generate first.

The platform's business value lies in compressing what would normally be days of fragmented, manual cross-functional review into a process that completes in minutes, without sacrificing the rigor of specialized domain analysis or the transparency of a visible reasoning trail. Every recommendation is traceable, every disagreement is recorded, and every confidence score is explicit.

Architecturally, ShadowBoard AI is built as a production-shaped system from the outset — authenticated, observable, asynchronous, and deployed on public cloud infrastructure — rather than a throwaway prototype. This positions the platform to grow directly from a hackathon demonstration into a genuinely deployable decision-intelligence product, with a clear path toward additional agents, deeper enterprise integrations, and richer analytics as outlined above.

ShadowBoard AI represents a practical, working example of AI agent collaboration applied to a problem that matters: helping organizations make better, faster, and more transparent decisions.